

1. ソフトウェア工学の概論						
_____とは、人、機械、コンピュータが互いに協調してある_____のことである。	情報システム	_____は、____、____、____の頭文字を取ったものである。ソフトウェアの良し悪しはプロジェクトの成否に直結し、規模の増大により計画どおりに進みづらくなる傾向がある。	QCD			
	目的を達成する仕組み全体		品質			
	ソフトウェア	_____とは、コストなどの制約のもと、ソフトウェアを開発・利用するための_____である。その目的は、ソフトウェア開発に関わる問題を_____することにある。経験則に基づき、コンピュータの性能向上により情報システムが構築され始めた結果、開発の QCD などが問題となった。これに関連して、_____という法則があり、「遅れている開発プロジェクトでは、開発要員の増員は遅れをひどくする」と述べている。また、_____は_____と_____の積で表される。現代では、性能やメモリ利用効率を落としても、読みやすく修正しやすいコードが必要とされている。ソフトウェアモデルはプログラミング言語より抽象的に作成され、開発方法や開発環境も重要となる。コードやドキュメントの再利用やパターンを使うことで資産を生かすことが可能である。	コスト			
	プログラム		納期			
	データ					
	ドキュメント					
「_____」は人間の「_____」ものであり、相手は非専門家であることが多いため、「_____」や「_____」が重要となる。一方、「_____」は「_____」ものであり、相手も専門家であることが多いため、「_____」や「_____」が重要である。	情報システム	ソフトウェア開発における_____は、顧客から要求を聞き要求事項としてまとめる「_____」と、コードを記述するための方法を定義する「_____」からなる。一方、_____は「_____」と「_____」の工程を指す。	ソフトウェア工学			
	業務を支える		理論等を整理したもの			
	使いやすさ		解決・改善			
	わかりやすさ		ブルックスの法則			
	組込みシステム		工数			
	物理現象に働きかける		人数			
	安全性		期間			
	リアルタイム性					
ソフトウェアは対象次第での作り変えなどにより、_____になる。また、要求が増加し様々な機能が追加されていくことで、____、_____している。さらに、ビジネスモデルの変化や技術の更新に適応しなければならず、_____という特徴がある。	多種多様		上流工程			
	大規模化		要求定義			
	複雑化		設計			
	変化が激しい		下流工程			
			実装			
			テスト			
		不具合の多くは_____にて生まれるが、その原因は_____にて生まれることが多い。この場合、上流工程からやり直す「_____」が発生し、工期に影響する。管理や組織の問題として、_____は時間、コスト、資源が限られている活動であり、_____は_____の管理を指す。規模の増大により、調整を行う管理業務の重要性が増し、コミュニケーション力は重要なスキルとなる。変化の問題として、ソフトウェアは再利用されることが多く、再利用のためにソースを追加・編集していくと、結果として読みづらくなる。業務の中核を支える「_____」は、長期間の修正の繰り返しで品質が劣化し、これをソフトウェアは「_____」を持つと表現する。				
		2. 情報システムとソフトウェア				
		_____は、事実や概念、指示の_____であり、例えば文字コードが挙げられる。一方、_____は、データを評価して得られる_____のことである。	データ			
			表現			
			情報			
			意味			
		ソフトウェアの動作は_____である。これは CPU が CLK で飛び飛びの値しか表現できないためである。また、OS がプログラムを切り替えながら動作するように、ソフトウェアは_____（一時停止）して動作する。最終的に機械語で実行され、インタリーブも機械語単位で行われる。	離散的			
			インタリーブ			
		_____では、クライアントがサーバにアクセスし、クライアントの数が多い傾向がある。処理方式には、1 台のサーバで処理を行う「_____」と、複数のサーバに処理を分担させる「_____」がある。集中処理は効率的でデータの一貫性を保てるが、故障に弱く負荷が集中しやすい。分散処理は負荷軽減やバックアップが可能だが、データの一貫性やコストに課題がある。	クライアントサーバ方式			
			集中処理			
			分散処理			

問題 1

1. ソフトウェア工学の概論					
_____とは、人、機械、コンピュータが互いに協調してある_____のことである。		_____は、_____、_____、_____の頭文字を取ったものである。ソフトウェアの良し悪しはプロジェクトの成否に直結し、規模の増大により計画どおりに進みづらくなる傾向がある。		不具合の多くは_____にて生まれるが、その原因は_____にて生まれることが多い。この場合、上流工程からやり直す「_____」が発生し、工期に影響する。管理や組織の問題として、_____は時間、コスト、資源に限られている活動であり、_____は_____の管理を指す。規模の増大により、調整を行う管理業務の重要性が増し、コミュニケーション力は重要なスキルとなる。変化の問題として、ソフトウェアは再利用されることが多く、再利用のためにソースを追加・編集していくと、結果として読みづらくなる。業務の中核を支える「_____」は、長期間の修正の繰り返しで品質が劣化し、これをソフトウェアは「_____」を持つと表現する。	
_____は、_____、_____、および関連する_____から構成される。		_____とは、コストなどの制約のもと、ソフトウェアを開発・利用するための_____である。その目的は、ソフトウェア開発に関わる問題を_____することにある。経験則に基づき、コンピュータの性能向上により情報システムが構築され始めた結果、開発のQCDなどが問題となった。これに関連して、_____という法則があり、「遅れている開発プロジェクトでは、開発要員の増員は遅れをひどくする」と述べている。また、_____は_____と_____の積で表される。現代では、性能やメモリ利用効率を落としても、読みやすく修正しやすいコードが必要とされている。ソフトウェアモデルはプログラミング言語より抽象的に作成され、開発方法や開発環境も重要となる。コードやドキュメントの再利用やパターンを使うことで資産を生かすことが可能である。		2. 情報システムとソフトウェア	
「_____」は人間の「_____」ものであり、相手は非専門家であることが多いため、「_____」や「_____」が重要となる。一方、「_____」は「_____」ものであり、相手も専門家であることが多いため、「_____」や「_____」が重要である。		ソフトウェア開発における_____は、顧客から要求を聞き要求事項としてまとめる「_____」と、コードを記述するための方法を定義する「_____」からなる。一方、_____は「_____」と「_____」の工程を指す。		_____は、事実や概念、指示の_____であり、例えば文字コードが挙げられる。一方、_____は、データを評価して得られる_____のことである。	
ソフトウェアは対象次第での作り変えなどにより、_____になる。また、要求が増加し様々な機能が追加されていくことで、_____、_____している。さらに、ビジネスモデルの変化や技術の更新に適応しなければならず、_____という特徴がある。				ソフトウェアの動作は_____である。これはCPUがCLKで飛び飛びの値しか表現できないためである。また、OSがプログラムを切り替えながら動作するように、ソフトウェアは_____（一時停止）して動作する。最終的に機械語で実行され、インターリーブも機械語単位で行われる。	
				_____では、クライアントがサーバにアクセスし、クライアントの数が多い傾向がある。処理方式には、1台のサーバで処理を行う「_____」と、複数のサーバに処理を分担させる「_____」がある。集中処理は効率的でデータの一貫性を保てるが、故障に弱く負荷が集中しやすい。分散処理は負荷軽減やバックアップが可能だが、データの一貫性やコストに課題がある。	

システム構築方式にはいくつか種類がある。_____は1台のコンピュータでシステムを構築するため低コストだが、故障するとシステムが停止する。_____は_____と_____の2系統でシステムを構築し、通常は別々の処理を実行するが、故障時は従系を使うことで故障に強い。しかし高コストである。_____は2台のコンピュータで同じ処理を行うため、高コストで効率は悪い。	シンプレックスシステム	3. 要求定義	_____は、「顧客が欲しいモノ」から「_____」をまとめる工程である。そのプロセスは、_____、_____、_____、そして_____からなる。_____とは、問題を解決したり目標を達成するために顧客が必要とする条件や能力のことで、_____と_____に分けられる。機能要求はソフトウェアが提供するサービスを指し、非機能要求は品質や制約、法令要求などを指すが、厳密な区別はない。システムによって影響を受ける、または与える人、グループ、組織を_____と呼び、ステークホルダごとに要求の重要度は異なる。要求仕様書をまとめることは非常に難しい作業である。	要求定義	要求分析の手法として、目標を設定し認識・整理する「_____」がある。これは直接満たすのが難しい場合にサブゴールを設定し、下位がすべて満たされると上位も満たされる「_____」や、下位のうち1つでも満たされると上位も満たされる「_____」を用いる。また、ゴールよりも具体的に記述する「_____」では、一連の出来事を時系列に記述し、システム利用の流れや振る舞いの理解に有用である。シナリオの記述方法には、日常的な言葉で文章として書く「_____」、箇条書きや表で表現する「_____」、UMLのシーケンス図やアクティビティ図に沿ってモデルを描く「_____」がある。さらに、システムの利用のされ方を記述し、システムが提供する機能をグループ化できる「_____」も用いられる。これらの方法はどれか一つに絞る必要はない。	ゴール分析
	デュプレックスシステム			要求仕様書		AND 分割
	主系			要求獲得		OR 分割
専用ハードウェアであるため、ハードウェアへの依存が強く、_____が悪い。イベントが発生してからそれに応じて応答する_____、時間制約を守る必要のある_____、_____、継続して機能し続ける性質である_____、そして人・モノ・環境に損害を与えない性質である_____が求められる。	従系	要求分析	要求仕様化	検証	シナリオ分析	
	デュアルシステム	検証	要求	機能要求	叙述的なシナリオ	
		要求	非機能要求	ステークホルダ	構造的なシナリオ	
					ソフトウェアモデルの利用	
					ユースケース分析	
	移植性		_____では顧客から情報収集を行う。その手法として、深掘りできるが情報が未整理になりがちな「_____」、発展性がない「_____」、複数のステークホルダで議論し気づきにくい要求を獲得しうる「_____」がある。また、発想支援のために「_____」や「_____」も用いられる。ステークホルダ間で利害が一致しない場合は、「落とし所」を見つける必要がある。	要求獲得	_____は、要求分析した結果を整理・構造化して「ソフトウェアへの要求」にする工程である。顧客から技術者へ誤解なく伝える必要があり、ソフトウェア開発の入力であるため、厳密で一般的な記述が求められる。要求仕様が満たすべき性質として、満たすべき要求のみを記述する「_____」、要求が一意に解釈できる「_____」、すべての要求が記述されている「_____」、要求間に矛盾がない「_____」、要求の優先順位が明確な「_____」、要求がテスト可能である「_____」、要求が容易に変更できる「_____」、そして根拠が明確である「_____」が挙げられる。	要求仕様化
	リアクティブ性			インタビュー		正当性
	リアルタイム性			アンケート		非あいまい性
	資源の制約			ワークショップ		完全性
	信頼性			ブレインストーミング		無矛盾性
安全性	プロトタイピング		順序付け	検証可能性	変更可能性	追跡可能性

問題 2

システム構築方式にはいくつか種類がある。_____は1台のコンピュータでシステムを構築するため低コストだが、故障するとシステムが停止する。_____は_____と_____の2系統でシステムを構築し、通常は別々の処理を実行するが、故障時は従系を使うことで故障に強い。しかし高コストである。_____は2台のコンピュータで同じ処理を行うため、高コストで効率は悪い。		3. 要求定義		要求分析の手法として、目標を設定し認識・整理する「_____」がある。これは直接満たすのが難しい場合にサブゴールを設定し、下位がすべて満たされると上位も満たされる「_____」や、下位のうち1つでも満たされると上位も満たされる「_____」を用いる。また、ゴールよりも具体的に記述する「_____」では、一連の出来事を時系列に記述し、システム利用の流れや振る舞いの理解に有用である。シナリオの記述方法には、日常的な言葉で文章として書く「_____」、箇条書きや表で表現する「_____」、UMLのシーケンス図やアクティビティ図に沿ってモデルを描く「_____」がある。さらに、システムの利用のされ方を記述し、システムが提供する機能をグループ化できる「_____」も用いられる。これらの方法はどれか一つに絞る必要はない。		
専用ハードウェアであるため、ハードウェアへの依存が強く、_____が悪い。イベントが発生してからそれに応じて応答する____、時間制約を守る必要のある____、____、継続して機能し続ける性質である____、そして人・モノ・環境に損害を与えない性質である_____が求められる。						

4. 設計			
_____は、「要求仕様書」から「_____」を作成する工程である。ここでは、ソフトウェアの_____、構成要素とその関係や特性を決定する。具体的には、外部設計とも呼ばれる「_____」の決定、内部設計（アーキテクチャ設計）とも呼ばれる構成要素とその関係を決める「_____」の決定、そしてハードウェアやOS、使用技術を決める「_____」の決定が含まれる。設計の確認も行われ、プログラムの実現はデータ構造とアルゴリズムによって行われる。	設計	_____は、モジュール内の結びつきの強さを指し、強度が強い（役割が強い）ほど良いとされる。その種類を強い順に挙げると、モジュールが1つの機能のみ提供する「_____」、特定データへのアクセスを行う機能をまとめた「_____」、手順的に共通データを参照・変更する「_____」、順番に実行される複数の機能を手順として1つのモジュールにする「_____」、ある時点で実行する機能がまとまっている「_____」、複数機能で1つが選択可能な「_____」、そして無秩序に機能を寄せ集めた「_____」がある。	モジュール強度
	設計書		機能的
	構造		情動的
	外部インターフェース		連絡的
	内部の実現方法		手順的
要求が_____を定義するのに対し、設計は_____を定義する。要求は設計に対応する必要があり、設計は品質に影響を与える。プラットフォームは制約となり、要求に対して設計を割り当てると新たな問題が発生する可能性がある。	What	_____は、モジュール間の結びつきの強さを指し、結合度が弱い（関係があまりない）ほうが良いとされる。その種類を強い順に挙げると、他のモジュール内の構成要素に直接アクセスする「_____」、複数のモジュール間で読み書き可能な構造化されたデータ領域を参照する「_____」、複数のモジュール間で読み書き可能な変数を参照している場合の「_____」、他のモジュールの特定の制御を目的としてif文などの条件式で使用される引数を使う「_____」、共有変数で構造体データを受け取る場合の「_____」、共有変数でない変数を引数とする場合の「_____」がある。モジュール間で情報隠蔽が重要であり、公開された操作のみを許可する。これを実現するのが「_____」である。	時間的
	How		論理的
			暗号的
	モジュール		モジュール結合度
	モジュール化		内容結合
_____はソフトウェアの構成要素である。_____とは、モジュールとその関係性を定義することであり、その良し悪しは品質に直結する。モジュールは作業分担の単位にもなり、再利用できる構造にするためには_____が良い状態である必要がある。適切な規模に分割することが重要である。	モジュール性	_____は、_____（設計図）やインスタンスをモジュール化の単位とする（独立性の向上が目的）設計手法である。クラスはフィールド（データ）とメソッド（振る舞い）を定義する。_____に基づき、setterとgetterを介してのみフィールドへのアクセスが許可される。_____はスーパークラスの持つフィールドとメソッドをサブクラスにもたせる機能である。_____では、サブクラスでスーパークラスのメソッドを再定義できる（オーバーライド）。_____だが、渡されるデータの型が異なっても型に応じた処理を実行できる「_____」という機能もある。	共通結合
			外部結合
			制御結合
			スタンプ結合
			データ結合
_____は、_____, _____, _____の3つの制御構造を用いて記述される。これは実行速度を多少遅くしてでも、可読性を重視する考え方である。	構造化プログラミング	_____（プロセス）は入力データを出力データへ変換する。_____は、システム全体を1つの機能として、その機能を細分化することである。最初は抽象的に、段階的に具体的な機能分割を行う「_____」が用いられる。設計手法には、データ同士の関係を定義する「_____」があり、詳細化の面では機能分割に近い。また、イベントと現在の状態で振る舞いと次の状態が決まるリアクティブシステムにおいて用いられる「_____」では、ステートマシン図（状態遷移図）で遷移の全体像を直感的に捉え、_____で考慮漏れがないか確認しやすい。	カプセル化
	逐次		
	分岐		
	反復		
		オブジェクト指向設計	機能
		クラス	機能分割
		カプセル化	段階的詳細化
		継承	情報・データに注目する設計
		多態性	状態に注目する設計
		余談	状態遷移表
		オーバーロード	

問題 3

4. 設計					
_____は、「要求仕様書」から「_____」を作成する工程である。ここでは、ソフトウェアの_____、構成要素とその関係や特性を決定する。具体的には、外部設計とも呼ばれる「_____」の決定、内部設計（アーキテクチャ設計）とも呼ばれる構成要素とその関係を決める「_____」の決定、そしてハードウェアや OS、使用技術を決める「_____」の決定が含まれる。設計の確認も行われ、プログラムの実現はデータ構造とアルゴリズムによって行われる。		_____は、モジュール内の結びつきの強さを指し、強度が強い（役割が強い）ほど良いとされる。その種類を強い順に挙げると、モジュールが 1 つの機能のみ提供する「_____」、特定データへのアクセスを行う機能をまとめた「_____」、手順的に共通データを参照・変更する「_____」、順番に実行される複数の機能を手順として 1 つのモジュールにする「_____」、ある時点で実行する機能がまとまっている「_____」、複数機能で 1 つが選択可能な「_____」、そして無秩序に機能を寄せ集めた「_____」がある。		_____（プロセス）は入力データを出力データへ変換する。_____は、システム全体を 1 つの機能として、その機能を細分化することである。最初は抽象的に、段階的に具体的な機能分割を行う「_____」が用いられる。設計手法には、データ同士の関係を定義する「_____」があり、詳細化の面では機能分割に近い。また、イベントと現在の状態で振る舞いと次の状態が決まるリアクティブシステムにおいて用いられる「_____」では、ステートマシン図（状態遷移図）で遷移の全体像を直感的に捉え、_____で考慮漏れがないか確認しやすい。	
要求が_____を定義するのに対し、設計は_____を定義する。要求は設計に対応する必要があり、設計は品質に影響を与える。プラットフォームは制約となり、要求に対して設計を割り当てると新たな問題が発生する可能性がある。		_____は、モジュール間の結びつきの強さを指し、結合度が弱い（関係があまりない）ほうが良いとされる。その種類を強い順に挙げると、他のモジュール内の構成要素に直接アクセスする「_____」、複数のモジュール間で読み書き可能な構造化されたデータ領域を参照する「_____」、複数のモジュール間で読み書き可能な変数を参照している場合の「_____」、他のモジュールの特定の制御を目的として if 文などの条件式で使用される引数を使う「_____」、共有変数で構造体データを受け取る場合の「_____」、共有変数でない変数を引数とする場合の「_____」がある。モジュール間で情報隠蔽が重要であり、公開された操作のみを許可する。これを実現するのが「_____」である。		_____は、_____（設計図）やインスタンスをモジュール化の単位とする（独立性の向上が目的）設計手法である。クラスはフィールド（データ）とメソッド（振る舞い）を定義する。_____に基づき、setter と getter を介してのみフィールドへのアクセスが許可される。_____はスーパークラスの持つフィールドとメソッドをサブクラスにもたせる機能である。_____では、サブクラスでスーパークラスのメソッドを再定義できる（オーバーライド）。_____だが、渡されるデータの型が異なっても型に応じた処理を実行できる「_____」という機能もある。	
_____はソフトウェアの構成要素である。_____とは、モジュールとその関係性を定義することであり、その良し悪しは品質に直結する。モジュールは作業分担の単位にもなり、再利用できる構造にするためには_____が良い状態である必要がある。適切な規模に分割することが重要である。					
_____は、_____, _____, _____の 3 つの制御構造を用いて記述される。これは実行速度を多少遅くしてでも、可読性を重視する考え方である。					